A Retry-Payout Guarantee for Asset Locks v2

Hilawe Semunegus and Joel Valenzuela (TheDesertLynx)

Proposed requirement for the Asset Locks v2 specification listed in the June 2026 Dash Core Group development update. One page, one ask, submitted while the draft is still in flight, which is the cheapest moment a guarantee like this will ever have.

Plain words. When credits leave Platform, the money arrives on the Core chain as a special withdrawal transaction with no inputs, already signed by a masternode quorum. Exchanges still wait about 2.5 minutes for a mined, ChainLocked block before crediting it, because of one loose end in the rules. A withdrawal that expires unmined can be retried, and nothing promises the retry pays the same place the same amount, or that a retry happens at all. Close that loose end with one short paragraph in Asset Locks v2 and wallets can credit withdrawals the moment they appear, with no Core consensus change and no new network message. With shielded pools scheduled to activate on or around July 12, 2026 per the July 9 development update, withdrawal speed is the protocol-side missing piece of an instant, private cash-out, and this is its cheapest fix.

What the rules say today

Under DIP-27, an asset unlock transaction is special transaction type 9. It has zero inputs, a withdrawal index that must be unique, an explicit miner fee, and a signature from the Platform validator quorum. Nodes refuse it once the chain height exceeds its signing height by 48 blocks, it is not eligible for InstantSend, and it becomes final by inclusion in a ChainLocked block.

Authorization is therefore never the receiver's problem. A valid unlock in the mempool already carries quorum consent, and with no inputs there are no outpoints to double-spend. The only events that can change what the receiver ultimately gets are expiry followed by a retry, and abandonment. DIP-27 says an expired withdrawal "can be retried" and stops there. Nothing in the specification binds a retried transaction to the original payout, requires a retry to happen, or defines an abandoned state a receiver could detect.

The requirement

Add one normative guarantee to Asset Locks v2, in three clauses:

> Platform fixes the payout script and amount for a withdrawal index at the moment it accepts the withdrawal request. Every quorum signing for that index MUST commit to exactly that payout script and amount. Signings for the same index may otherwise differ only in signing height, quorum, signature, and miner fee, and a fee adjustment MUST NOT be taken from the payout. A withdrawal index whose transaction expires unmined MUST be re-signed until a transaction for it is mined.

The last clause is the load-bearing one, and it is deliberately the strong form. An accepted withdrawal must be a promise to pay, not a promise to try. If the specification instead permits an abandoned terminal state, then credit extended before mining is provisional rather than final, a receiver who credited a withdrawal and later sees its index abandoned is holding a loss, and cautious

receivers will keep waiting for ChainLocked inclusion, which defeats the purpose. Should abandonment be unavoidable for some edge case, it must be explicit, detectable by receivers (for example through the existing asset unlock status query surface), and documented as making pre-mining credit provisional in that case.

The payout clauses may already describe how the implementation behaves. In that case v2 only needs to write the behavior down as a guarantee rather than an accident of the current code. The fee clause is called out separately because DIP-27 itself names low Core fees as a reason an unlock may fail to be mined, so retries plausibly need fee headroom.

One forward-looking note, so the guarantee survives the intended end state. Today Platform emits exactly one withdrawal per asset unlock transaction (one index, one output), a deliberate simplification for the accelerated Platform release, even though Core already allows up to 32 outputs per asset unlock transaction. The end-state design, reported by the Platform team, batches several concurrent withdrawals into one transaction to reduce fees and space. The guarantee should be written to cover that. Read "the payout for an index" as the full, fixed set of outputs bound to that index, and add that once a batch is formed it is re-signed as an immutable unit under a stable index, a withdrawal is not silently re-batched under a different index on retry. With that rule a receiver still credits on mempool arrival and deduplicates by index, locating its own output in the batch. If instead a withdrawal may move between batches on retry, the binding has to follow the individual withdrawal rather than the index, which is a stronger requirement worth stating explicitly.

What it buys

- Instant acceptance becomes pure receiving policy. A wallet or exchange whose own fully validating Core node holds a valid quorum-signed unlock in its mempool can credit it immediately, because every signing for that index pays out identically and the guarantee obliges Platform to keep re-signing until one of them mines. This finality claim rests on the strong liveness form. Under an abandonment fallback the same credit would be provisional, which is why the request asks for the strong form. Receivers MUST deduplicate credits by withdrawal index rather than by transaction id, since a re-signing changes the txid while paying the same recipient.
- Binding the payout to the index at request time, rather than to a "first signing", also removes any ambiguity from concurrent signings across quorum rotation. Any two valid transactions for the same index pay identically by construction, so the receiver never needs to know which signing came first or which will confirm.
- No Core consensus change and no new Core network message. Enforcement lives where withdrawals are created, in Platform's state transition and signing rules, which is exactly the surface Asset Locks v2 is already revising. This is not merely cosmetic, a source audit (below) finds the current implementation already preserves the payout across re-signings, but by an immutable-document implementation property rather than a consensus invariant, so codifying it in v2 is what turns today's behavior into a guarantee integrators can rely on. The heavier alternative, a defensive Core-side check that rejects a same-index payout conflict, or a deterministic Core lock message binding index, txid, and payout, only becomes necessary if the Platform-side guarantee is judged insufficient.
- The user-facing effect is a withdrawal that clears in seconds instead of minutes, arriving exactly when shielded balances make withdrawals the flagship flow.

Scope, so this does not read as asking Core to trust Platform. Instant acceptance is a receiver-side decision that credits on a signature-valid mempool unlock and trusts the Platform quorum, the same class of decision InstantSend crediting already is. It does not ask Core to weaken the credit-pool withdrawal limit, which is enforced at mining and is Core's circuit breaker against a compromised or buggy Platform, nor to treat an unmined unlock as chainable, since the unlock's txid is not final until mined and it is not guaranteed to mine at all. Core stays protected when the happy path fails. The ask is a Platform-side guarantee that makes acceptance safer, not a weakening of Core's self-protection.

Asks

- A source audit of dashpay/platform and dashpay/dash at pinned commits (companion document d0-audit-retry-payout.md) finds the current implementation already preserves the payout script and amount across unlock re-signings. Confirm that reading and codify it, rather than leaving it an implementation accident.
- State whether re-signings may adjust the miner fee, and confirm a fee change is never taken from the payout.
- Add the guarantee above, or its equivalent, to the Asset Locks v2 draft as a normative requirement, with retry-until-mined as the liveness rule. If any abandonment path must exist, make it explicit, receiver-detectable, and documented as demoting pre-mining credit to provisional.
- State in the same section whether receivers may treat a valid mempool unlock as payout-final and deduplicate by withdrawal index, so exchange integrators have one paragraph to point at.

Context

This requirement is the load-bearing question of project D1 (formerly P1 and P7) in dash-ideas-scoping.md, where the full analysis lives, including the fallback lock-message design for the case where the guarantee cannot hold, the mempool-eviction and fee questions the audit should also answer, and the child-transaction and light-client extensions that can be layered on later. Withdrawal speed is the protocol-side piece of private cash-out. The wallet-side piece, protecting the pool boundary from amount and timing correlation, is project D2 of the same document.